

Laurea Magistrale in Ingegneria Elettronica

Programmazione di Sistemi IOT

Sistema di Allarme Smart

Studente

Tarantino Danilo Antonio
Matricola 247705

Professore

Gravina Raffaele

A.A. 2024/2025

Introduzione

I sensori di allarme smart a microcontrollori wireless a basso consumo sono fondamentali per sistemi di sicurezza moderni. Sfruttando la connettività wireless e una gestione energetica efficiente, questi sensori possono essere posizionati liberamente senza vincoli di cablaggio, monitorando costantemente l'ambiente (es. movimento, apertura porte/finestre, temperatura, luminosità, umidità) per rilevare anomalie. La loro utilità risiede nella capacità di inviare tempestivamente notifiche e dati in caso di pericolo e/o anomalie, operando per lunghi periodi con batterie e integrandosi facilmente in ecosistemi smart home, aumentando significativamente l'efficacia e la flessibilità dei sistemi di allarme.



Descrizione Progetto

Questo elaborato ha come scopo la presentazione di un sistema di allarme smart, progettato per attivarsi quando vengono rilevati movimenti o condizioni ambientali (temperatura, umidità e luminosità) anomale. Esso si articola in 3 stadi:

Sensore PIR (HC-SR501) + Telosb 1 = Sensor Node per l'acquisizione dei dati e la trasmissione radio;

Telosb 2 = BaseStation per ricevere i dati, visualizzarli a schermo in tempo reale e inviare messaggi;

Telosb 3 = Actuator per attivare un buzzer in caso di allarme per rilevato movimento.

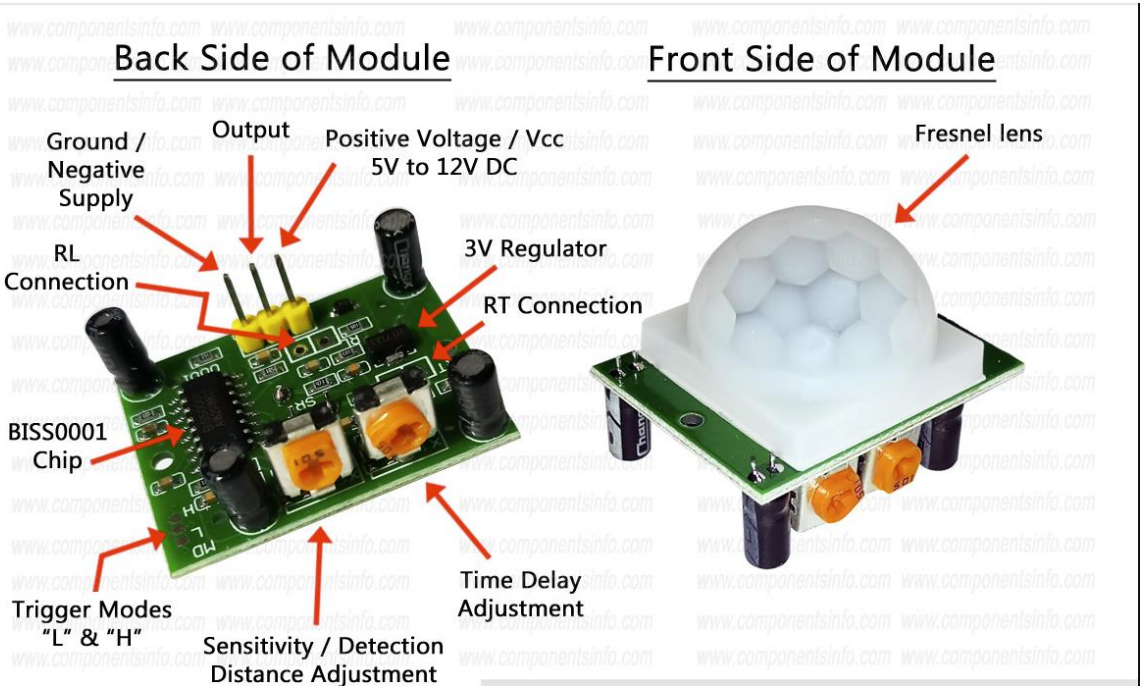
Sensor Node

Il Telosb 1 viene alimentano sulla power rail 3.3V mentre il sensore PIR sulla 5V. Nel circuito descritto, i resistori fungono da partitore di tensione, un elemento circuitale fondamentale per l'interfacciamento del sensore PIR con l'ingresso analogico-digitale (ADC0) del TelosB. La loro funzione consiste nell' adattamento del livello di tensione. Il partitore riduce la tensione di uscita del sensore PIR, portandola a un livello compatibile con l'intervallo operativo dell'ADC del TelosB. Con un Resistore R1 da 2 kΩ collegato tra l'uscita del PIR e il nodo di misurazione e un Resistore R2 da 5.1 kΩ tra il nodo di misurazione e la massa, la tensione di 5V in uscita dal PIR viene ridotta a circa 3.59V secondo la formula del partitore di tensione : $V_{out(adc0)} = V_{in(PIR)} \times R2/(R1+R2)$. Il partitore previene l'applicazione di tensioni eccessive al pin ADC0, salvaguardando il microcontrollore da potenziali danni o letture saturate. Dopo aver rilevato il movimento in sensore PIR invia al Telosb un segnale sulla porta adc0 e questo viene inoltrato a sua volta mediante comunicazione radio al Telosb che opera da Basestation.

Componenti



SENSORE PIR HC-SR501



1)- Nodo Sensore



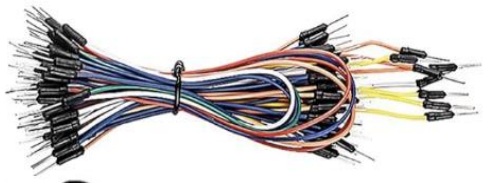
BREADBOARD



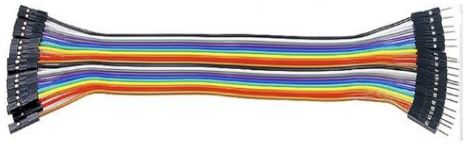
CIRCUITO DI ALIMENTAZIONE MB-102



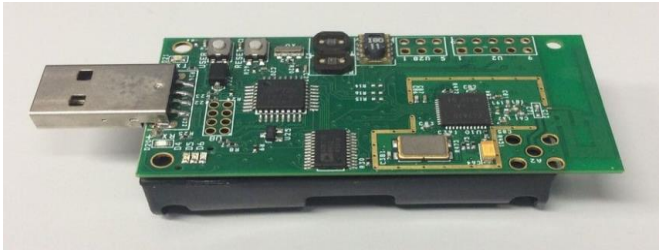
2X RESISTORI



CAVI BREADBOARD M-M

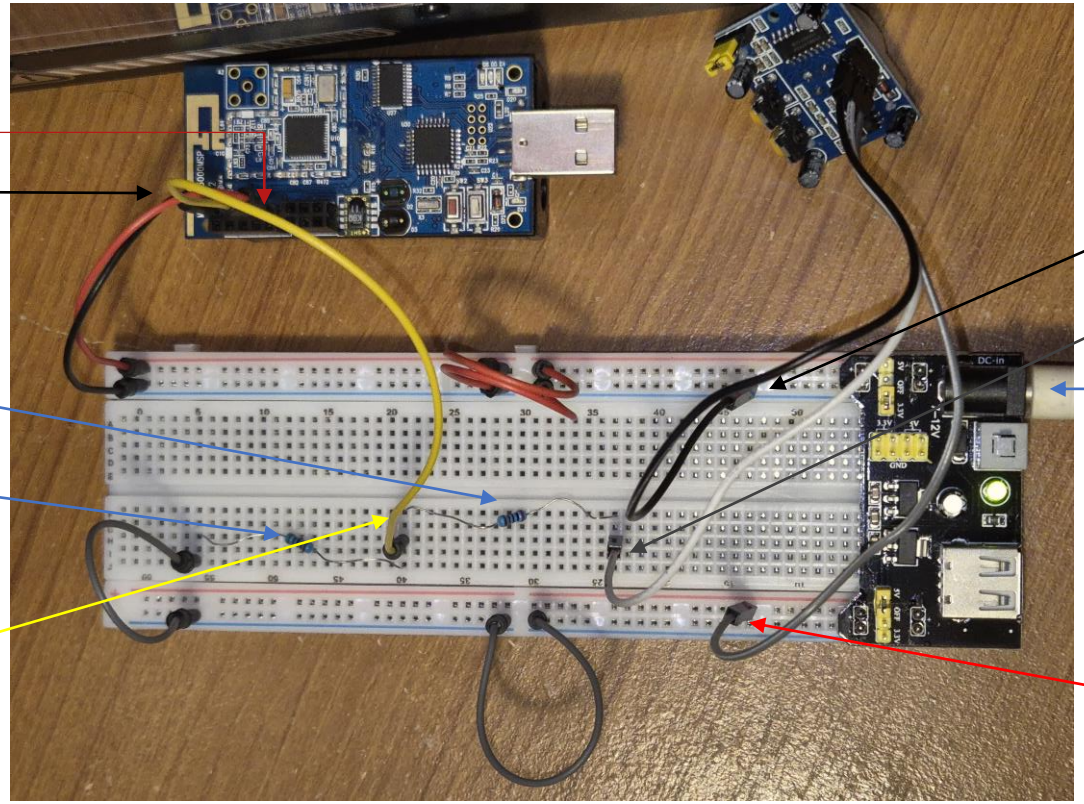


CAVI JUMPER M-F



1X TELOSB

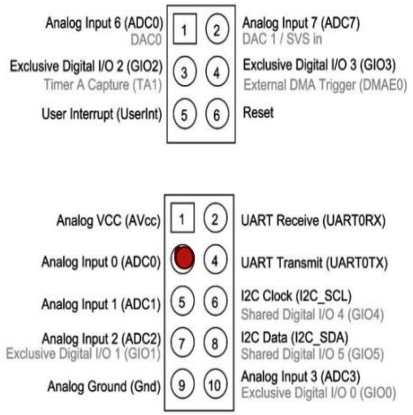
Connessioni fisiche



ADC

Questo modulo è il cervello di configurazione per il Convertitore Analogico-Digitale (ADC) del microcontrollore MSP430 nel TelosB. Il suo ruolo è garantire che ogni lettura analogica sia accurata, affidabile e sicura per il microcontrollore.
Il seguente modulo definisce esattamente come l'ADC deve leggere un sensore analogico, impostando parametri cruciali come:

Canale;
tensioni di riferimento;
Velocità;
Precisione;
Frequenza;
tempo di campionamento del segnale.




```
#include "Msp430Adc12.h"

generic module AdcConfigC(uint8_t channel) {
    provides interface AdcConfigure<const msp430adc12_channel_config_t*>;
}

implementation {
    const msp430adc12_channel_config_t adcConfig = {
        inch: channel,
        sref: REFERENCE_AVcc_AVss,
        ref2_5v: REFVOLT_LEVEL_NONE,
        adc12ssel: SHT_SOURCE_SMCLK,
        adc12div: SHT_CLOCK_DIV_1,
        sht: SAMPLE_HOLD_16_CYCLES,
        sampcon_ssel: SAMPCON_SOURCE_SMCLK,
        sampcon_id: SAMPCON_CLOCK_DIV_1
    };

    async command const msp430adc12_channel_config_t* AdcConfigure.getConfiguration() {
        return &adcConfig;
    }
}
```

Header

```
#ifndef PKT_H
#define PKT_H

enum {
    AM_COMBINED_SENSOR_MSG = 7
};

typedef nx_struct CombinedSensorMsg {
    nx_uint16_t nodeid;
    nx_uint8_t motionDetected;
    nx_uint16_t temperatureRaw;
    nx_uint16_t humidityRaw;
    nx_uint16_t lightRaw;
    nx_uint8_t alarmStatus;
    nx_uint8_t motionAlarmActive;
    nx_uint8_t lightAlarmActive;
    nx_uint8_t tempAlarmActive;
    nx_uint8_t humAlarmActive;
    nx_uint8_t baseStationError;
} CombinedSensorMsg;

enum {
    MOTION = 1,
    NO_MOTION = 0
};

enum {
    ALARM_OFF = 0,
    ALARM_ON = 1
};

#endif
```

Questo file definisce la struttura esatta di ogni messaggio che il nodo sensore invia e che la Base Station riceve e si assicura che tutti i dispositivi parlino la stessa "lingua", garantendo una comunicazione wireless chiara e senza errori.

Struttura

```
#include "pkt.h"

configuration CombinedSensorAppC { }
implementation {
    components MainC, CombinedSensorC, LedsC;
    components new TimerMilliC() as SensorTimerC;
    components ActiveMessageC;
    components new AMSenderC(AM_COMBINED_SENSOR_MSG);

    components new SensirionSht11C() as TemperatureDriver;
    components new SensirionSht11C() as HumidityDriver;
    components new HamamatsuS1087ParC() as LightDriver;

    components new AdcReadClientC() as Adc0ReaderC;
    components new AdcConfigC(INPUT_CHANNEL_A0) as Adc0Config;

    CombinedSensorC.Boot -> MainC.Boot;
    CombinedSensorC.Leds -> LedsC;
    CombinedSensorC.SensorTimer -> SensorTimerC;

    CombinedSensorC.AMSend -> AMSenderC;
    CombinedSensorC.Packet -> AMSenderC;
    CombinedSensorC.RadioControl -> ActiveMessageC;

    CombinedSensorC.Temperature -> TemperatureDriver.Temperature;
    CombinedSensorC.Humidity -> HumidityDriver.Humidity;
    CombinedSensorC.Light -> LightDriver;

    CombinedSensorC.Adc0Sensor -> Adc0ReaderC.Read;
    Adc0ReaderC.AdcConfigure -> Adc0Config;
}
```

Interconnessione di tutti i componenti sw e hw necessari. Dichiarazione dei moduli presenti nel sistema e definizione interfacce.

Logica

```
#include "msp430.h"
#include "pkt.h"

#define SAMPLING_INTERVAL 1024
#define ADC_MOTION_THRESHOLD 2000

module CombinedSensorC @safe() {
    uses interface Boot;
    uses interface Timer<TMilli> as SensorTimer;
    uses interface Leds;
    uses interface AMSend;
    uses interface Packet;
    uses interface SplitControl as RadioControl;

    uses interface Read<uint16_t> as Temperature;
    uses interface Read<uint16_t> as Humidity;
    uses interface Read<uint16_t> as Light;

    uses interface Read<uint16_t> as Adc0Sensor;
}
```

```
implementation {
    message_t packet;
    bool locked = FALSE;

    uint16_t pirAdcValue;
    uint16_t currentTempRaw;
    uint16_t currentHumidityRaw;
    uint16_t currentLightRaw;

    bool pirReadDone = FALSE;
    bool tempReadDone = FALSE;
    bool humReadDone = FALSE;
    bool lightReadDone = FALSE;

    void attemptSend();
```

```
event void RadioControl.startDone(error_t err) {
    if (err == SUCCESS) {
        call SensorTimer.startPeriodic(SAMPLING_INTERVAL);
        call Leds.led0On();
    } else {
        call RadioControl.start();
    }
}
```

```
    event void SensorTimer.fired() {
        if (!locked) {
            pirReadDone = FALSE;
            tempReadDone = FALSE;
            humReadDone = FALSE;
            lightReadDone = FALSE;

            call Adc0Sensor.read();
            call Temperature.read();
            call Humidity.read();
            call Light.read();
        }
    }
```

1)- Nodo Sensore

```
event void Adc0Sensor.readDone(error_t result, uint16_t data) {
    if (result == SUCCESS) {
        pirAdcValue = data;
        pirReadDone = TRUE;
    } else {
        pirReadDone = FALSE;
    }
    attemptSend();
}

event void Temperature.readDone(error_t code, uint16_t data) {
    if (code == SUCCESS) {
        currentTempRaw = data;
        tempReadDone = TRUE;
    } else {
        tempReadDone = FALSE;
    }
    attemptSend();
}

event void Humidity.readDone(error_t code, uint16_t data) {
    if (code == SUCCESS) {
        currentHumidityRaw = data;
        humReadDone = TRUE;
    } else {
        humReadDone = FALSE;
    }
    attemptSend();
}

event void Light.readDone(error_t code, uint16_t data) {
    if (code == SUCCESS) {
        currentLightRaw = data;
        lightReadDone = TRUE;
    } else {
        lightReadDone = FALSE;
    }
    attemptSend();
}
```

```
void attemptSend() {
    if (pirReadDone && tempReadDone && humReadDone && lightReadDone && !locked) {
        CombinedSensorMsg* msgPayload;

        msgPayload = (CombinedSensorMsg*)call Packet.getPayload(&packet, sizeof(CombinedSensorMsg));

        if (msgPayload == NULL) {
            return;
        }

        msgPayload->nodeid = TOS_NODE_ID;
        if (pirAdcValue > ADC_MOTION_THRESHOLD) {
            msgPayload->motionDetected = MOTION;
        } else {
            msgPayload->motionDetected = NO_MOTION;
        }
        msgPayload->temperatureRaw = currentTempRaw;
        msgPayload->humidityRaw = currentHumidityRaw;
        msgPayload->lightRaw = currentLightRaw;
        msgPayload->alarmStatus = ALARM_OFF;

        if (call AMSend.send(AM_BROADCAST_ADDR, &packet, sizeof(CombinedSensorMsg)) == SUCCESS) {
            locked = TRUE;
            call Leds.led0Toggle();
        } else {
            locked = FALSE;
        }
    }
}
```

```
event void AMSend.sendDone(message_t* bufPtr, error_t error) {
    if (&packet == bufPtr) {
        locked = FALSE;
    }
    if (error == SUCCESS) {
        call Leds.led0Off();
    } else {
        call Leds.led0On();
    }
}
```

Comunicazione

Sensor Node => Base Station:

Comunicazione Radio Wireless (Nodo-Nodo) mediante chip radio CC2420 integrato nei TelosB che adotta nativamente lo standard IEEE 802.15.4. Esso definisce i livelli più bassi della pila protocollare:

Livello Fisico che gestisce la modulazione, la trasmissione e la ricezione dei segnali radio;

Livello di Controllo dell'Accesso al Mezzo (MAC), responsabile dell'accesso condiviso al mezzo radio, la gestione dei frame e l'indirizzamento.

Protocollo Applicativo: TinyOS impiega in protocollo Active Messages (AM). Esso facilita lo scambio di messaggi strutturati tra i nodi, utilizzando un ID di messaggio specifico per il multiplexing dei dati.

Base Station

La Base Station ha il compito fondamentale di elaborare i dati grezzi ricevuti via wireless dal nodo sensore, convertendoli in misurazioni fisiche leggibili per la visualizzazione e l'analisi.

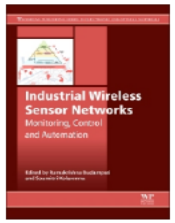
Per la temperatura e l'umidità, la Base Station elabora i dati grezzi provenienti dal sensore misto SHT11. Vengono applicate formule polinomiali con coefficienti specifici forniti dalla documentazione tecnica, convertendo efficacemente le letture grezze in gradi Celsius e umidità relativa.

Per la luminosità riceve i dati grezzi dal fotodiodo Hamamatsu S1087-01. Questo sensore, che cattura lo spettro visibile e infrarosso, fornisce un output digitale a 12 bit dopo una conversione ADC che utilizza un riferimento di tensione di 1.5V e un circuito con una resistenza di 100 kΩ. Applicando la formula di conversione basata su questi parametri, i dati grezzi vengono trasformati in valori di luminosità in Lux. La formula calcola i Lux moltiplicando il dato grezzo ricevuto per un coefficiente di circa **2.816** ottenuto dal seguente calcolo:

$$\frac{0.769 \times 10^6 \times 1.5\text{V} \times 1000}{4096 \times 100000\ \Omega}$$

Conversione dati

- From the book:



Industrial Wireless Sensor Networks

1st Edition

Monitoring, Control and Automation

☆☆☆☆☆ Write a review

Editors: R Budampati, S Kolavennu

Hardcover ISBN: 9781782422303

eBook ISBN: 9781782422372

AG family of SHT1x temperature/humidity surface mount. SHT11 integrates analog measuring devices and signal processing logic to provide calibrated output. Moisture is measured by a capacitive sensor and the temperature of a band-gap (proportional to absolute temperature) sensor. Measurements are then converted to 14-bit digital words which are conveyable to TinyOS via the serial interface. Converting the digital value to relative humidity (RH) units is achieved using the following formula:

$$RH_{linear} = c_1 + c_2 \times SO_{RH} + c_3 \times SO_{RH}^2$$

where SO_{RH} represents the 12-bit measured data (Sensirion Output), $c_1 = -4$, $c_2 = 0.0405$, $c_3 = -2.8 \times 10^{-6}$. The respective conversion formula for temperature measurements is the following:

$$Temp = c_1 + c_2 \times SO$$

Light sensors for TinyOS motes typically use photo-diodes manufactured by Corporation. Hamamatsu S1087 photo-diodes detect photosynthetically active radiation (PAR) and S1087-01 photo-diodes detect all the visible spectrum, including infrared (TSR). Photosynthetic active radiation can be defined as the electromagnetic spectrum of visible light, namely wavelengths from 400 to 700 nm that helps process plant growth via photosynthesis. Both analog sensors TSR and PAR convert measurements to 12-bit digital words of length using 1.5-V signal pulses. LEDs generate current I along a resistance 100 kΩ. The output of S1087 or S1087-01 can be converted into units for brightness, LUX, using the following formulas:

$$LUX = 0.625 \times 1e6 \times I \times 1000 \text{ for photo} - \text{diode S1087}$$

$$LUX = 0.769 \times 1e6 \times I \times 1000 \text{ for photo} - \text{diode S1087} - 01$$

- note: 1e6 stands for 1million

where I is defined as:

$$I = \frac{AD_{output} \times \left(\frac{1.5}{4096}\right)}{10{,}000}$$

The microcontroller MPS30 has internal sensors such as temperature and voltage mentioned in the description of the devices category TELOS. The analog measurements of the sensor voltage are converted into digital words of length 12 bits and conversion to physical units is carried by the following formula:

$$VCC = 2 \times \left(\frac{AD_{output}}{4096}\right) \times V_{ref}$$

where $V_{ref} = 1.5$ V.

Struttura

```
#define NEW_PRINTF_SEMANTICS
#include "printf.h"
#include "pkt.h"

configuration BaseStationAppC { }
implementation {
    components MainC, BaseStationC, LedsC;
    components ActiveMessageC;
    components new AMReceiverC(AM_COMBINED_SENSOR_MSG);
    components new AMSenderC(AM_COMBINED_SENSOR_MSG) as AlarmSenderC;

    components PrintfC;
    components SerialStartC;

    BaseStationC.Boot -> MainC;
    BaseStationC.Leds -> LedsC;

    BaseStationC.AMReceive -> AMReceiverC;
    BaseStationC.RadioControl -> ActiveMessageC;

    BaseStationC.AlarmSender -> AlarmSenderC;
    BaseStationC.Packet -> AlarmSenderC;
}
```

Interconnessione di tutti i componenti sw e hw necessari. Dichiarazione dei moduli presenti nel sistema e definizione interfacce.

Logica

```
#define NEW_PRINTF_SEMANTICS

#include "printf.h"
#include "pkt.h"

module BaseStationC @safe() {
    uses interface Boot;
    uses interface Leds;
    uses interface Receive as AMReceive;
    uses interface Packet;
    uses interface SplitControl as RadioControl;
    uses interface AMSend as AlarmSender;
}
```

```
implementation {
    #define T_THRESHOLD 40.0
    #define H_THRESHOLD 75.0
    #define L_THRESHOLD 200.0

    message_t alarmPacket;
    bool alarmSendLocked = FALSE;
```

```
event void Boot.booted() {
    call RadioControl.start();
}

event void RadioControl.startDone(error_t err) {
    if (err == SUCCESS) {
        call Leds.led1On();
    } else {
        call RadioControl.start();
    }
}
```

2)- Base Station

```
event message_t* AMReceive.receive(message_t* bufPtr, void* payload, uint8_t len) {
    CombinedSensorMsg* msg;
    uint16_t dataCelsius;
    float c1, c2, c3, humidityLinear, lux;
    uint16_t humidityPrinted;
    uint16_t luxPrinted;

    bool isAlarmConditionMet = FALSE;
    bool motionAlarmTriggered = FALSE;
    bool lightAlarmTriggered = FALSE;
    bool tempAlarmTriggered = FALSE;
    bool humAlarmTriggered = FALSE;


    msg = (CombinedSensorMsg*)payload;

    printf("\n Dati ricevuti dal Nodo Sensore (ID: %u) \n", msg->nodeid);

    if (msg->motionDetected == MOTION) {
        printf(" ALLARME : MOVIMENTO RILEVATO !!!\n");
        isAlarmConditionMet = TRUE;
        motionAlarmTriggered = TRUE;
    } else {
        printf("  nessun movimento rilevato... \n");
    }

    dataCelsius = -39 + (uint16_t)(0.01 * msg->temperatureRaw);
    printf("  Temperatura: %d C\n", dataCelsius);
    if (dataCelsius > T_THRESHOLD) {
        printf(" ALLARME : Temperatura Elevata ! \n");
        isAlarmConditionMet = TRUE;
        tempAlarmTriggered = TRUE;
    }
```

```
c1 = -4.0;
c2 = 0.0405;
c3 = -2.8e-6;
humidityLinear = c1 + c2 * msg->humidityRaw + c3 * msg->humidityRaw * msg->humidityRaw;
humidityPrinted = (uint16_t)(humidityLinear * 100);
printf("  Umidita': %u.%02u%%\n", humidityPrinted / 100, humidityPrinted % 100);

if (humidityLinear > H_THRESHOLD) {
    printf(" ALLARME : Umidita' Elevata !\n");
    isAlarmConditionMet = TRUE;
    humAlarmTriggered = TRUE;
}


lux = (float)msg->lightRaw * 2.8161;


printf("  Luminosita': %u LUX\n", (uint16_t)lux);
if (lux > L_THRESHOLD) {
  printf(" ALLARME: Luminosita' Elevata !\n");
        isAlarmConditionMet = TRUE;
        lightAlarmTriggered = TRUE;
}
```

2)- Base Station

```
if (!alarmSendLocked) {
    CombinedSensorMsg* alarmPayload = (CombinedSensorMsg*)call Packet.getPayload(&alarmPacket, sizeof(CombinedSensorMsg));

    if (alarmPayload != NULL) {
        alarmPayload->nodeid = TOS_NODE_ID;
        alarmPayload->motionDetected = msg->motionDetected;
        alarmPayload->temperatureRaw = msg->temperatureRaw;
        alarmPayload->humidityRaw = msg->humidityRaw;
        alarmPayload->lightRaw = msg->lightRaw;

        alarmPayload->alarmStatus = isAlarmConditionMet ? ALARM_ON : ALARM_OFF;
        alarmPayload->motionAlarmActive = motionAlarmTriggered ? ALARM_ON : ALARM_OFF;
        alarmPayload->lightAlarmActive = lightAlarmTriggered ? ALARM_ON : ALARM_OFF;
        alarmPayload->tempAlarmActive = tempAlarmTriggered ? ALARM_ON : ALARM_OFF;
        alarmPayload->humAlarmActive = humAlarmTriggered ? ALARM_ON : ALARM_OFF;

        if (call AlarmSender.send(AM_BROADCAST_ADDR, &alarmPacket, sizeof(CombinedSensorMsg)) == SUCCESS) {
            alarmSendLocked = TRUE;
            printf(" ALLARME %s (Movimento: %u, Luce: %u, Temperatura: %u, Umidita': %u) \n",
                (isAlarmConditionMet ? "ON" : "OFF"),
                alarmPayload->motionAlarmActive,
                alarmPayload->lightAlarmActive,
                alarmPayload->tempAlarmActive,
                alarmPayload->humAlarmActive);
        }
    }
}
printfflush();
return bufPtr;
}
```

```
event void AlarmSender.sendDone(message_t* bufPtr, error_t error) {
    if (&alarmPacket == bufPtr) {
        alarmSendLocked = FALSE;
    }
    if (error != SUCCESS) {
        printf(" ERRORE: Invio allarme fallito (%d)\n", error);
        printfflush();
    }
}
```

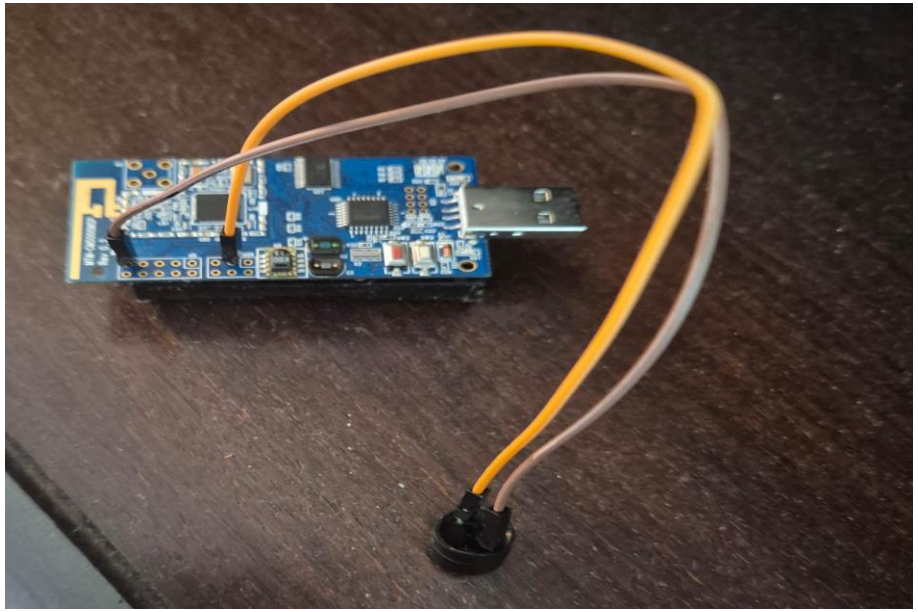
Comunicazione

BaseStation => Host PC:

Per la trasmissione di dati a un computer host (debug e/o visualizzazione), il TelosB impiega l'interfaccia UART (Universal Asynchronous Receiver/Transmitter). I dati vengono inviati tramite la porta seriale USB del TelosB. Il livello di Trasporto sul computer host, si avvale della applicazione Serial Forwarder per leggere i dati dalla porta seriale e renderli disponibili su una connessione TCP. Questo permette ad altre applicazioni (es. dashboard, gateway) di connettersi via rete per ricevere e elaborare i dati seriali.

Attuatore

Il nodo attuatore, realizzato con un telosb alimentato a doppia batteria AA, ha il compito di ricevere messaggi via radio dalla Base Station e attivare un buzzer in caso di rilevato movimento. Se rilevate determinate condizioni di allarme, accende il led rosso per segnalare una luminosità eccessiva (apertura porte o finestre), e il led blu per segnalare il rilevato movimento (intrusione), oppure entrambe le condizioni se accesi contemporaneamente.



Struttura

```
#include "pkt.h"

configuration ActuatorAppC { }
implementation {
    components MainC, ActuatorC, LedsC;
    components ActiveMessageC;

    components new AMReceiverC(AM_COMBINED_SENSOR_MSG);

    components HplMsp430GeneralIOC;

    ActuatorC.Boot -> MainC;
    ActuatorC.Leds -> LedsC;

    ActuatorC.AMReceive -> AMReceiverC;
    ActuatorC.RadioControl -> ActiveMessageC;


    ActuatorC.OutputPin -> HplMsp430GeneralIOC.Port23;
}
```

Collego l'interfaccia ActuatorC.OutputPin a HplMsp430GeneralIOC.Port20. Questo permette al software di manipolare direttamente il pin fisico P2.3 (GIO2), alzando la sua tensione a 3.3V per attivare il buzzer quando l'allarme è attivo, e abbassandola a 0V per disattivarlo.

Logica

```
#include "msp430.h"
#include "pkt.h"

module ActuatorC @safe() {
    uses interface Boot;
    uses interface Leds;
    uses interface Receive as AMReceive;
    uses interface SplitControl as RadioControl;
    uses interface HplMsp430GeneralIO as OutputPin;
}

implementation {
    event void Boot.booted() {
        call RadioControl.start();
        call Leds.led0Off();
        call Leds.led1Off();
        call Leds.led2Off();
        call OutputPin.makeOutput();
        call OutputPin.clr();
    }

    event void RadioControl.startDone(error_t err) {
        if (err == SUCCESS) {
            call Leds.led1On();
        } else {
            call RadioControl.start();
        }
    }

    event void RadioControl.stopDone(error_t err) {
        call Leds.led1Off();
        call Leds.led0Off();
        call Leds.led2Off();
        call OutputPin.clr();
    }
```

```
    event message_t* AMReceive.receive(message_t* bufPtr, void* payload, uint8_t len) {
        if (len == sizeof(CombinedSensorMsg)) {
            CombinedSensorMsg* msg = (CombinedSensorMsg*)payload;

            if (msg->motionAlarmActive == ALARM_ON) {
                call Leds.led2On();
                call OutputPin.set();
            } else {
                call Leds.led2Off();
                call OutputPin.clr();
            }

            if (msg->lightAlarmActive == ALARM_ON) {
                call Leds.led0On();
            } else {
                call Leds.led0Off();
            }
        }
        return bufPtr;
    }
}
```

Visualizzazione Dati elaborati

Per facilitare il debugging e il monitoraggio in tempo reale dei dati e dello stato del sistema, la Base Station è configurata per inviare i suoi output di debugging tramite la porta seriale del TelosB.

Questo flusso di dati è reso accessibile sulla rete tramite il Serial Forwarder. Esso agisce come un ponte, inoltrando tutti i dati ricevuti dalla porta seriale fisica del TelosB a una porta di rete (TCP/IP) specificata. In tal caso la 9002

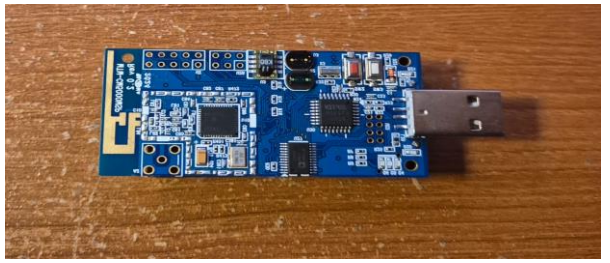
Qualsiasi applicazione client, come un terminale o uno script, può connettersi a localhost:9002 (o all'indirizzo IP del computer Host) per visualizzare l'output.

In questo modo, è possibile monitorare il comportamento della Base Station e del flusso di dati ricevuto dai nodi sensore in tempo reale, rendendo il processo di sviluppo e debug molto più efficiente.

Per la visualizzazione utilizzo il PrintfClient di Tiny OS:

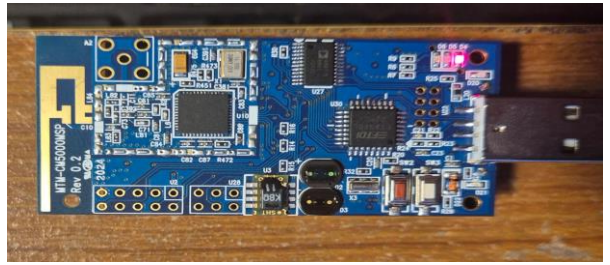
```
Dati ricevuti dal Nodo Sensore (ID: 1)
 nessun movimento rilevato...
 Temperatura: 29 C
 Umidita': 44.11%
 Luminosita': 61 LUX
ALLARME OFF (Movimento: 0, Luce: 0, Temperatura: 0, Umidita': 0)
```

Condizione di allarme OFF



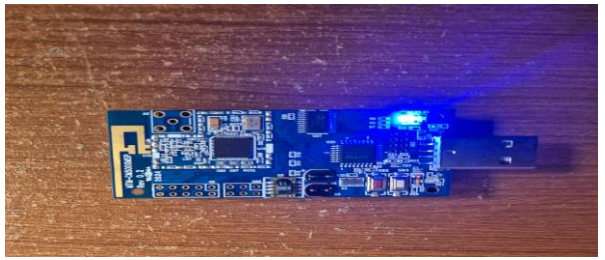
```
Dati ricevuti dal Nodo Sensore (ID: 1)
 nessun movimento rilevato...
 Temperatura: 29 C
 Umidita': 44.18%
 Luminosita': 1461 LUX
ALLARME: Luminosita' Elevata !
ALLARME ON (Movimento: 0, Luce: 1, Temperatura: 0, Umidita': 0)
```

Condizione di allarme ON
a causa di luminosità
elevata (rilevata apertura
porte e/o accensione luce)



```
Dati ricevuti dal Nodo Sensore (ID: 1)
ALLARME : MOVIMENTO RILEVATO !!!
 Temperatura: 29 C
 Umidita': 44.11%
 Luminosita': 64 LUX
ALLARME ON (Movimento: 1, Luce: 0, Temperatura: 0, Umidita': 0)
```

Condizione di allarme ON
a causa di rilevato movimento



Grazie per l'attenzione